

Logic-VLA: A Temporal Logic Conditioned Vision-Language-Action Model

Abstract—Vision-language-action (VLA) models can follow natural-language (NL) task instructions, but such instructions may not precisely specify safety-critical or spatiotemporal requirements on the resulting behavior. We introduce *Logic-VLA*, a formal-requirement-aware VLA that conditions on Signal Temporal Logic (STL) specifications supplied at inference time. Logic-VLA uses a syntax-graph-based STL encoder pre-trained to capture temporal logic semantics. Policy adaptation proceeds in two stages: STL-conditioned supervised fine-tuning on satisfying demonstrations is followed by trajectory-level preference optimization over matched satisfying–violating rollout pairs using a flow-matching surrogate for Identity Preference Optimization. This formulation improves formal requirement satisfaction while preserving the nominal NL task. We evaluate Logic-VLA in closed-loop quadcopter navigation simulation across randomized photorealistic environments, including STL formulas unseen during training, and on a physical mobile robot under deployment-relevant STL requirements. In simulation, Logic-VLA improves STL satisfaction over an STL-blind base policy by 24.8 to 40.7 percentage points (pp) while reducing nominal NL task success by at most 1.8 pp. On the physical robot, Logic-VLA satisfies the specified STL formulas in 97.6% of the retained trials. These results demonstrate that a single VLA can adapt its behavior to varying formal requirements without requiring a separate policy for each specification. Our code will be made publicly available.

I. INTRODUCTION

A longstanding goal in robotics is to develop agents that flexibly perform diverse tasks in complex environments from intuitive human instructions. Vision-language-action (VLA) models [1], [2] have made significant progress toward this goal by grounding rich visual and language representations in action generation. Given an observation and a natural-language instruction, a VLA produces actions for manipulation, navigation, and other embodied tasks. However, a task instruction alone may not fully specify all desired properties of the resulting behavior. For instance, a robot may need to maintain a prescribed clearance from an object, visit regions in a particular order, complete part of a task within a deadline, or satisfy one condition until another occurs. Such safety-critical, spatiotemporal, and reactive requirements can be difficult to express unambiguously and evaluate precisely using natural-language alone. Signal Temporal Logic (STL) [3], [4], broadly applied in robotics [5], provides a formal language for specifying such trajectory-level requirements together with robust semantics for evaluating their satisfaction.

This raises a new challenge for VLA models. A deployment-time formal requirement should not replace the natural-language (NL) task, nor should each new requirement require training a separate policy. We seek a single policy whose behavior can adapt to a formal requirement supplied

at inference time while retaining the NL task-following capability of a pre-trained VLA. We study *requirement-aware post-training*: given a pre-trained VLA, we adapt the policy to jointly condition on visual observations, the NL task, and an STL specification. Unlike post-training toward a fixed preference or reward, the desired formal requirement is provided as a policy input and may vary across deployments.

To this end, we propose Logic-VLA, a temporal logic-conditioned VLA together with a general post-training framework for incorporating formal trajectory requirements into pre-trained policies. Our instantiation builds on $\pi_{0.5}$ [1] and uses a syntax-graph STL encoder built on TeLoGraF [6] to map formal specifications into the VLA conditioning space. We then introduce a two-stage post-training procedure that first imitates STL-satisfying demonstrations and subsequently applies trajectory-level Identity Preference Optimization (IPO) [7] to prefer satisfying executions over the violating alternatives. At inference time, Logic-VLA jointly conditions on the visual observation, the NL task, and a potentially unseen STL specification, allowing a single policy to adapt its behavior to varying formal requirements without retraining for each specification. While our implementation uses $\pi_{0.5}$, the post-training procedure can be applied to other flow-matching VLA policies and provides a general template for requirement-conditioned VLA adaptation. We summarize our contributions:

- We introduce Logic-VLA, a $\pi_{0.5}$ -based temporal logic conditioned VLA that fuses structured formal requirements with multimodal VLA inputs.
- We develop a two-stage requirement-aware post-training framework combining STL-conditioned supervised fine-tuning (SFT) with trajectory-level preference optimization over matched satisfying–violating executions.
- We evaluate Logic-VLA in closed-loop quadcopter navigation across seen and unseen STL specifications and demonstrate its deployment on a physical mobile robot.

II. RELATED WORK

Vision-Language-Action Models: Vision-language-action (VLA) models couple pre-trained vision-language representations with visuomotor action generation [1], [8]. These models have demonstrated strong semantic generalization across manipulation and navigation tasks. Natural-language, however, primarily describes *what* behavior is desired and need not precisely specify *how* it should be executed (e.g. route choices or safety margins). Recent work has therefore investigated safety-aware VLA training and evaluation [9]–[11]. Control barrier functions

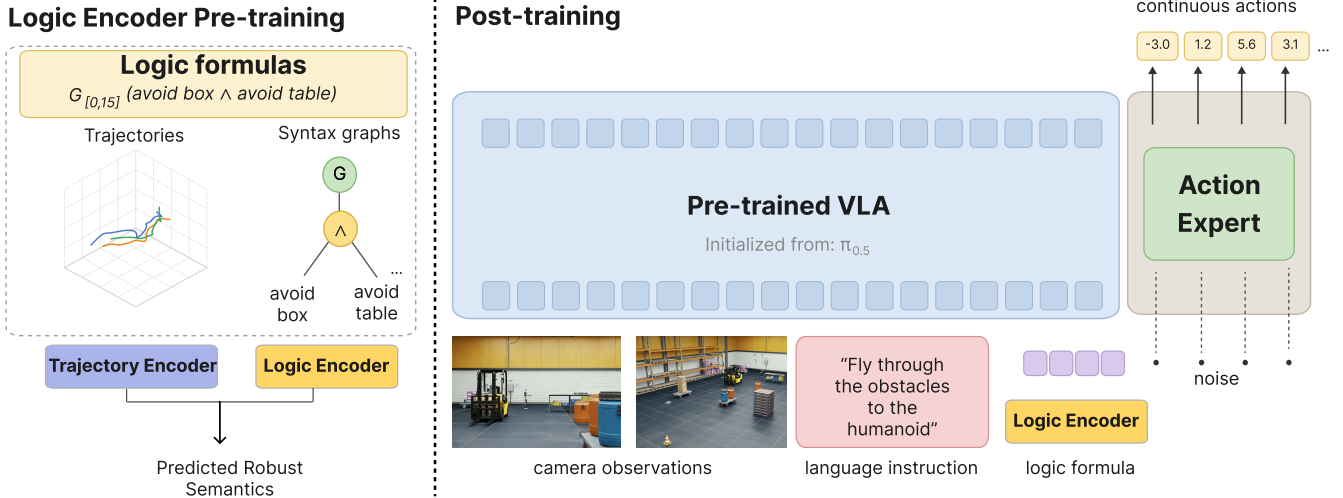


Fig. 1: **Overview of Logic-VLA.** Logic-VLA augments a pre-trained VLA with a structured logic conditioning pathway so that the policy jointly follows a natural-language task and a formal temporal logic requirement. The logic encoder is pre-trained from trajectory-formula pairs using robust semantics supervision (Section IV-B). We integrate the encoder with a VLA in a post-training procedure (Section IV-C) including 1) supervised fine-tuning using logic satisfying demonstrations and 2) trajectory-level preference optimization using matched satisfying-violating trajectory pairs.

[12], [13] act as safety filters on VLA outputs. In contrast, we study formal-requirement-conditioned VLAs, where a temporal logic specification is provided as an additional policy input and may vary across deployments.

VLA Post-training: Supervised fine-tuning (SFT) adapts pre-trained robot policies by imitating demonstrations, but it does not explicitly distinguish between different-quality executions of the same task. Preference optimization [14], [15] provides a mechanism for learning from such comparisons. Direct Preference Optimization (DPO) [16] optimizes a policy directly from preferred and dispreferred samples relative to a reference policy, while Identity Preference Optimization (IPO) [7] provides an alternative pairwise objective designed to mitigate overfitting associated with preference-model assumptions. Preference-based post-training has been extended to robot policies [17], [18]. GRAPE [19] performs trajectory-wise preference optimization for VLA models using stage-wise costs from vision-language models and demonstrates alignment toward task completion, safety, and efficiency. FlowPRO [20] develops preference optimization specifically for flow-matching VLAs and introduces explicit regularization toward the reference policy. Related approaches [10] use constrained reinforcement learning to align VLAs with safety objectives. In these methods, the alignment objective is used to determine the behavior learned during post-training. Our setting differs in that the desired execution requirement is itself an input to the policy and may generalize at test time, similar to [21]. We obtain supervision directly from temporal logic semantics. Post-training therefore does not bias the policy toward a single global notion of safety.

Planning and Learning with Signal Temporal Logic Signal Temporal Logic (STL) [3], [22] provides a language for specifying spatiotemporal properties of state trajectories and a robust semantics measuring the degree of satisfaction. STL has been incorporated into optimization-based

planning [23], [24]. Specification-conditioned reinforcement-learning methods [25] learn policies for temporal logic tasks. Recent work studies visual planning under logic formulas. Vision-based methods [26], [27] address specification-conditioned *trajectory generation*, whereas our goal is to adapt a vision-language policy that repeatedly observes and acts in closed loop while jointly conditioning on a natural-language task and an STL requirement. A complementary family of methods enforces formal requirements during deployment. STL robust semantics can guide generative-policy sampling through predicted future states [28], and hierarchical frameworks can use STL for planning, monitoring, and replanning [29]. Such approaches perform additional optimization or reasoning during execution. Time-varying Control Barrier Functions [30], [31] act as STL-aware safety filters for planners, but their construction remains in state-space with difficult recursive feasibility guarantee. We use STL during post-training as both a policy condition and a source of trajectory-level supervision, so that requirement-dependent behavior is represented directly by the learned VLA policy. Our work connects specification-conditioned control with VLA post-training: a single policy is adapted to preserve the NL task while responding to different temporal and geometric requirements at test time.

III. PRELIMINARY

Consider an off-the-shelf vision-language-action (VLA) model π_θ , parameterized by θ . Given a natural-language (NL) task l , at each decision time $t_d \in \mathbb{N}$, conditioned on a visual observation o_{t_d} , the policy samples a K_a -step action $\hat{a}^d \sim \pi_\theta(\cdot \mid o_{t_d}, l)$. At each control time $t \in \mathbb{N}$, an execution rule selects the applied action $\hat{a}_t$ from the VLA outputs available up to t . At each time t , a discrete-time system with state $s_t \in \mathbb{R}^n$ evolves under an unknown dynamics f to reach the next-time state $s_{t+1} := f(s_t, \hat{a}_t, w_t)$ where $w_t \in \mathcal{W}$ is some exogenous process disturbance from an unknown

distribution. The VLA is invoked iteratively to produce a trajectory $s := (s_0, s_1, \dots, s_H)$, where H is the termination time. We say that s is the trajectory rollout from π_θ .

Suppose the NL task l specifies that a drone must circle around a table. A well-trained VLA may perform this nominal task, yet task completion alone does not guarantee compliance with additional formal requirements. For instance, the drone may be required to remain at least a prescribed distance from the table or return to base before a deadline. Such requirements can be difficult to express and evaluate precisely using NL alone. We therefore consider deployment-time formal requirements expressed using Signal Temporal Logic (STL). Existing VLA models generally do not account for such requirements during pre-training (i.e., large-scale training on diverse vision-language-action datasets for general perception, reasoning, and control capabilities). We study how to incorporate formal requirements through post-training (i.e., adapting an already pre-trained policy using additional requirement-aware objectives) while preserving the capabilities of the pre-trained policy.

A. Signal Temporal Logic

Signal Temporal Logic (STL) is considered in [3], [4]. Given a discrete-time trajectory s and an STL formula ϕ , we let $(s, \tau_0) \models \phi$ denote that s satisfies ϕ at time τ_0 . Without loss of generality, we let $s \models \phi$ denote $(s, 0) \models \phi$ (i.e., s satisfies ϕ). The syntax of an STL formula follows

$$\phi := \text{True} \mid \pi^\mu \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 U_{[a,b]} \phi_2$$

where $\pi^\mu : \mathbb{R}^n \rightarrow \{\text{True}, \text{False}\}$ is a predicate where $\pi^\mu(s_t) := \text{True}$ if $\mu(s_t) \geq 0$ and $\pi^\mu(s_t) := \text{False}$ otherwise, where $\mu : \mathbb{R}^n \rightarrow \mathbb{R}$ is a user-specified function. Predicates are atomic elements of an STL and can be used to express spatial relations (e.g. $\mu(s_t) := \|s_t - O\|_2 - d_{\min}$ expresses obstacle avoidance, where O is some obstacle location and $d_{\min}$ is some clearance margin). The symbols $\neg$ and $\wedge$ are Boolean operators denoting *negation* (not) and *conjunction* (and) respectively. The symbol $U_{[a,b]}$ is the temporal operator *until* with $a \leq b$ and $a, b \in \mathbb{N}$, which encodes that ϕ_1 has to be true until ϕ_2 is true at some future time in $[a, b] \cap \mathbb{N}$. From these operators, we can derive the operators for *disjunction* by $\phi_1 \vee \phi_2 := \neg(\neg\phi_1 \wedge \neg\phi_2)$, for *implication* by $\phi_1 \implies \phi_2 := \neg\phi_1 \vee \phi_2$, for *eventually* by $F_{[a,b]}\phi := \text{True } U_{[a,b]} \phi$, and for *globally* by $G_{[a,b]}\phi := \neg F_{[a,b]}\neg\phi$. The semantics of STL are standard [3]. The robust semantics $\rho^\phi(s, \tau_0)$ of STL are defined in [4], [22], which evaluate how robustly ϕ is satisfied vs. violated. Importantly, $(s, \tau_0) \models \phi$ if $\rho^\phi(s, \tau_0) > 0$ and a larger $\rho^\phi(s, \tau_0)$ denotes that the trajectory is more satisfied [4]. We use $\rho^\phi(s)$ to denote $\rho^\phi(s, 0)$. Given s and τ_0 , one needs a sufficient termination $H := H^\phi$ to evaluate satisfaction, whose computation is discussed in [32].

B. Problem Formulation

We study how to post-train a pre-trained VLA for deployment-time STL requirements while preserving its NL task-following capability. Given a pre-trained VLA, which we call the base policy, we target its satisfaction of an STL requirement at test time. We formalize Problem 3.1.

Problem 3.1: Consider a well-trained¹ VLA model π_θ with an NL specification distribution $\mathcal{L}$ and environment distribution $\mathcal{E}$ in which the dynamics f operates. Design a post-train procedure $\tilde{\theta} = \mathcal{A}(\theta)$ such that, at inference time, $\pi_{\tilde{\theta}}(\cdot \mid o, l, \phi)$ jointly conditions on the observation o from environment $\mathcal{E}$, NL task $l \sim \mathcal{L}$, and STL requirement ϕ , and produces a trajectory s satisfying both $s \models \phi$ and l .

Problem 3.1 poses several challenges: 1) Post-training must avoid largely altering the base policy’s rollout distribution, which could degrade NL task performance. 2) Learning requirement-dependent behavior from structured STL formulas requires capturing their spatiotemporal semantics, which may be difficult to acquire reliably through SFT alone. 3) STL semantics are globally defined over long-horizon trajectories, and the optimization must avoid reducing a trajectory-level requirement to independent action-chunk preferences. To provide sufficient demonstrations for our post-training procedure $\mathcal{A}$, we consider the Assumption 3.2.

Assumption 3.2: We assume access to an in-domain dataset $D := \{p^{(1)}, \dots, p^{(N_D)}\}$, where $p^{(i)} := (s^{(i)}, a^{(i)}, l^{(i)}, o^{(i)})$ with $s^{(i)}$ denoting a trajectory sample, $a^{(i)}$ denoting the action sequence, $l^{(i)}$ denoting the corresponding NL specification, and $o^{(i)}$ denoting the observation sequence leading to $s^{(i)}$.

The purpose of Assumption 3.2 is to provide in-distribution datasets, from which an offline monitor enables efficient STL learning during the post-training stage (as we show in Section IV). Trivially, one can directly sample D from the dataset used in training the base model or directly rollout π_θ to collect data that are roughly in-distribution. The STL specification enforced at test-time must not conflict with the NL specification, in which case Problem 3.1 is infeasible.

Assumption 3.3: In Problem 3.1, the NL specification l is not in conflict with the STL requirement ϕ .

To address Problem 3.1, we propose Logic-VLA, the requirement-conditioned policy resulting from our post-training procedure $\mathcal{A}$. The model augments a pre-trained VLA with an STL conditioning pathway, while $\mathcal{A}$ determines how its parameters are adapted from θ to $\tilde{\theta}$.

IV. LOGIC-VLA

A. Logic-Conditioned Supervision Construction

Given the in-distribution dataset $D = \{p^{(i)}\}_{i=1}^{|D|}$ from Assumption 3.2, we construct the STL-conditioned supervision used for post-training. We generate a bank of candidate STL specifications and construct satisfying demonstrations and matched satisfying–violating trajectory pairs which provide supervision for the two post-training stages in Section IV-C.

a) *Candidate formula bank:* Let $\mathcal{P}_{\text{STL}}$ denote a distribution over well-formed STL formulas under the grammar introduced in Section III-A. A draw from $\mathcal{P}_{\text{STL}}$ specifies both the syntactic structure of the formula and its predicate and temporal parameters. We sample a finite candidate formula bank $\Phi := \{\phi^{(1)}, \dots, \phi^{(N_\Phi)}\}$ where $\phi^{(j)} \sim \mathcal{P}_{\text{STL}}$. At inference, we likewise consider an STL requirement $\phi \sim \mathcal{P}_{\text{STL}}$

¹The model generates rollouts that reasonably satisfy the NL tasks.

which need not belong to Φ . The goal is therefore to learn a specification-conditioned policy that generalizes from the formulas observed during post-training to unseen formulas drawn from the same specification distribution.

b) Satisfying demonstrations and preference pairs:

Our two-stage post-training procedure requires two complementary forms of supervision. Stage 1 learns specification-conditioned behavior by imitating executions that satisfy the given STL requirement, while Stage 2 requires paired satisfying and violating executions so that the policy can learn which behavior should be preferred under the same task and specification. Accordingly, given the rollout dataset D and candidate formula bank Φ , we use a monitor [33] to evaluate each specification on the collected trajectories. Robust semantics are used to identify executions that clearly satisfy or violate a specification, and rollouts are matched only when they share the same nominal NL task and sufficiently comparable environment and initial conditions. From the satisfying rollouts, we extract STL-conditioned action demonstrations $D^+ = \{(\xi_{i,j,\tau}, a_\tau^{(i)})\}$ where $\xi_{i,j,\tau} = (o_\tau^{(i)}, l^{(i)}, \phi^{(j)})$ is the STL-conditioned context where $\phi^{(j)} \in \Phi$ is the STL specification satisfied by rollout i and $a_\tau^{(i)}$ is the demonstrated action chunk extracted at action-window index τ . For stage 2, we construct the trajectory-level preference dataset $P = \{(p_i^+, p_i^-, l_i, \phi_i)\}$ where p_i^+ and p_i^- are matched rollouts of the same nominal task l_i and are under comparable environment and initial conditions, with p_i^+ clearly satisfying and p_i^- clearly violating the same STL ϕ_i .

B. Logic Encoder

One key design component is a logic encoder mapping an STL specification into the VLA token space for integration with multimodal sensory representations. Specifically, it encodes each formula as a graph-structured representation and is pre-trained using STL robust semantics to capture the predicate, temporal, and logical structure of the formulas.

a) STL encoder: We adopt the syntax-graph backbone of TeLoGraF [6]. An STL formula ϕ is represented as a directed graph $G_\phi = (V_\phi, E_\phi)$, whose nodes encode Boolean, temporal, and comparison operators, with edges directed from operands to their parent operators. Node features $\mathbf{x}_v$ encode the operator type and temporal bounds. For predicates, the features also include the scalar-signal identity g_q and threshold c . A predicate $g_q(s_t) \geq c$ has $\mu(s_t) = g_q(s_t) - c$; for $g_q(s_t) \leq c$, $\mu(s_t) = c - g_q(s_t)$. In navigation, for instance, g_q can represent the projection of the ego-agent position along a coordinate or simply the distance to an environment asset. Unlike TeLoGraF [6], which encodes object geometry in the STL graph and conditions trajectory generation on the initial state, our encoder represents only the symbolic requirement. States variables needed to evaluate a predicate, such as ego-agent position or its spatial relation to an object, may be included in the offline state trajectory s used to compute STL semantics, but are not explicit inputs to the VLA during policy post-training or deployment. Instead, the VLA grounds the symbolic requirement through its visual observations. Thus, the logic representation remains

independent of the particular scene configuration. We adopt TeLoGraF [6] for the syntax-graph encoder architecture, using child-to-parent GCN message passing followed by mean pooling for a formula-level representation $E_{\text{STL}}(\phi) := \text{MeanPool}(\text{GCN}(G_\phi, X_\phi))$, where X_ϕ stacks the node features $\{\mathbf{x}_v\}_{v \in V_\phi}$. The encoder representation $E_{\text{STL}}(\phi)$ is independent of the observation.

b) STL encoder pre-training: While the graph encoder preserves the syntactic structure of an STL formula, we further pre-train E_{STL} to capture the STL semantics. We construct a set of trajectory-formula pairs $S_{\text{pre}} \subseteq D \times \Phi$. For each pair $(p^{(i)}, \phi^{(j)}) \in S_{\text{pre}}$, we compute the robust semantics $\rho^{(i,j)} := \rho^{\phi^{(j)}}(s^{(i)})$. We use these pairs to train E_{STL} through a robust semantics prediction objective. An auxiliary trajectory encoder E_{traj} maps $s^{(i)}$ to a latent representation, which is combined with $E_{\text{STL}}(\phi^{(j)})$ by an MLP regression head g_ψ to make the prediction $\hat{r}^{(i,j)}$. We optimize

$$\mathcal{L}_{\text{pre}} = \mathbb{E}_{(p^{(i)}, \phi^{(j)}) \in S_{\text{pre}}} \left[H_\delta \left(\hat{r}^{(i,j)} - \tanh \left(\rho^{(i,j)} / c \right) \right) \right],$$

where H_δ is the Huber loss with threshold δ , and $c > 0$ is a scale parameter used to normalize the robust semantics. Predicting the robust semantics encourages the embedding to encode the predicate thresholds, temporal bounds, and logical composition that determine how a trajectory satisfies or violates a formula. After pre-training, we discard E_{traj} and g_ψ . The pretrained E_{STL} is then jointly fine-tuned with the policy during post-training. For downstream VLA conditioning, we project the pre-trained formula representation into a sequence of N_{spec} specification tokens, $Z_\phi := \text{reshape}(W_p E_{\text{STL}}(\phi) + b_p) \in \mathbb{R}^{N_{\text{spec}} \times d_c}$, where d_c is the hidden dimension of the VLA conditioning stream, and W_p and b_p are learned parameters. We append Z_ϕ to the VLM prefix after the image and NL tokens, where bidirectional attention allows multi-modal fusion.

C. Architecture and Two-Stage Post-training

We choose $\pi_{0.5}$ [1] as the backbone of Logic-VLA but remark that the post-train recipe can be used for other flow-matching policies. The latent representation Z_ϕ is appended into the vision-language prefix, while the underlying backbone architecture is unchanged and initialized with the pre-trained weights. The full architecture is shown in Figure 1. Starting from the semantically pre-trained STL encoder, we adapt the policy in two stages using the datasets curated in Section IV-A. *Stage 1* is an STL-conditioned SFT on the satisfying demonstrations D^+ , establishing specification-conditioned behavior through imitation. *Stage 2* performs trajectory-level preference optimization on the satisfying-violating pairs P , teaching the policy to prefer executions that satisfy the given STL requirement over comparable executions that violate it.

a) Stage 1: STL-conditioned supervised fine-tuning:

For a demonstration $(\xi, a) \in D^+$, where $\xi = (o, l, \phi)$ denotes the STL-conditioned context, and a is the demonstrated action chunk, we optimize the standard conditional flow-matching objective of the $\pi_{0.5}$ action expert [1]. Since D^+ contains action demonstrations from trajectories satisfying

their associated STL requirements, this stage adapts the policy to imitate satisfying behavior while conditioning on the STL formula. The STL encoder and VLA parameters are jointly fine-tuned, with no additional STL-specific loss. Let $\pi_{\theta_{\text{SFT}}}$ denote the resulting policy. We initialize Stage 2 as $\tilde{\theta} \leftarrow \theta_{\text{SFT}}$ and retain a frozen copy as the reference policy $\pi_{\text{ref}} = \pi_{\theta_{\text{SFT}}}$, used for preference learning in the Stage 2.

b) Stage 2: trajectory-level preference optimization:

Stage 2 trains on the matched satisfying-violating rollout pairs P . For pair i , let p_i^+ and p_i^- denote the satisfying and violating rollouts associated with the same NL task l_i and STL requirement ϕ_i . At temporal window τ , let $a_{i,\tau}^+$ and $a_{i,\tau}^-$ denote the action chunks extracted from p_i^+ and p_i^- respectively. We evaluate both candidates under the shared conditioning context $\xi_{i,\tau} := (o_{i,\tau}^+, l_i, \phi_i)$, where $o_{i,\tau}^+$ is the observation from the satisfying rollout.

We adopt Identity Preference Optimization (IPO) [7], which learns a finite-margin preference between preferred and rejected samples relative to a reference policy. For likelihood-based policies, this margin is expressed via reference-relative log-policy ratios. However, the $\pi_{0.5}$ action expert is parameterized by conditional flow matching, for which exact action likelihood ratios are computationally expensive to evaluate. Following the likelihood-ratio surrogate of [34], we therefore use differences in conditional flow-matching losses as a surrogate for the reference-relative log-policy ratios. Because STL satisfaction is a trajectory-level property rather than an action-chunk-level property, we aggregate the policy score across a collection W_i of temporal windows sampled from pair i . Let $\ell_{\text{FM}}(\nu; a_\tau, \xi_\tau, \gamma, \epsilon)$ denote the standard conditional flow-matching loss of policy parameters ν for action chunk a_τ under context ξ_τ , flow time γ , and noise ϵ . For $\nu \in \{\tilde{\theta}, \theta_{\text{SFT}}\}$, define the trajectory-level flow-matching loss

$$\bar{\ell}_{\nu,i}^\pm := \frac{1}{|W_i|} \sum_{\tau \in W_i} \ell_{\text{FM}}(\nu; a_{i,\tau}^\pm, \xi_{i,\tau}, \gamma_i, \epsilon_i).$$

The same (γ_i, ϵ_i) realization is used across the temporal windows and both current/reference evaluations of pair i . Following the flow-matching likelihood surrogate, we define the reference-relative trajectory scores $q_i^\pm = \bar{\ell}_{\theta_{\text{SFT}},i}^\pm - \bar{\ell}_{\tilde{\theta},i}^\pm$. The corresponding preference margin is $\Delta_i = \beta (q_i^+ - q_i^-)$, where $\beta > 0$ controls the scale of the preference margin. We optimize this margin toward a finite target using a finite-margin objective. A relative preference objective can increase the satisfying-violating margin even if the current policy becomes worse at fitting the satisfying rollout, provided that the violating rollout degrades more. To prevent this, we introduce a one-sided preferred-rollout anchor $A_i = \left[\bar{\ell}_{\tilde{\theta},i}^+ - \text{sg} \left(\bar{\ell}_{\theta_{\text{SFT}},i}^+ \right) \right]_+$, where $\text{sg}(\cdot)$ denotes stop-gradient. The anchor is active only when the current policy incurs a larger flow-matching loss on the satisfying rollout than the frozen Stage-1 reference policy. The complete Stage 2 objective is $\mathcal{L}_{\text{pref}} := \mathbb{E}_i [\ell_{\text{IPO},i} + \lambda A_i]$, where $\lambda \geq 0$ controls the strength of the preferred-rollout anchor. The preference term learns the STL-dependent ordering between satisfying



Fig. 2: **Simulation navigation tasks.** One example trajectory for each drone task: (1) packing table, (2) wet floor sign, (3) G1 humanoid, (4) forklift, (5) fly through the corridor to the orange barrel, and (6) cardboard boxes.

and violating executions, while the anchor preserves the satisfying behavior acquired during Stage 1.

V. EVALUATION

Our evaluations consist of photorealistic simulations of a quadcopter closed-loop planning benchmark and physical mobile robot navigation experiments.

A. Quadcopter Simulation

We organize our empirical evaluation in the quadcopter study around three research questions. **1) Q1: Requirement satisfaction and task preservation.** Can Logic-VLA improve satisfaction of STL requirements while preserving the nominal NL task compared to pure STL-conditioned imitation or robust semantics driven optimization? **2) Q2: Specification generalization.** Does the learned policy generalize to STL formulas unseen during post-training? **3) Q3: Ablative design choices.** What is the effect of pre-training the STL encoder and choosing syntax-graph STL encoding vs. prompting via the NL channel?

Environment and tasks: We instantiate and evaluate Logic-VLA in a closed-loop quadcopter navigation setting using NVIDIA Isaac Sim 5.1. We consider ten randomized photorealistic warehouse environments and six natural-language navigation tasks (shown in Figure 2) centered on reaching ordered target regions. We collect $|D| = 3000$ collision-free dynamically feasible reference trajectories spanning these environment-task combinations. We execute these trajectories with a DJI Mavic 2 Pro model in simulation and record the corresponding actions (absolute drone position) together with synchronized visual observations from both an egocentric onboard camera and a third-person room-view camera, which are provided to the policy as visual inputs in the navigation demonstrations. Our control procedure is via position update through Isaac Sim API calls, tracking the predicted positions. Starting from a pre-trained $\pi_{0.5}$ base checkpoint, we fine-tune the policy on the collected demonstrations D without STL conditioning. The resulting STL-blind navigation policy serves as the common initialization for all post-training baselines. We

TABLE I. **Quadcopter Study Evaluation Results.** Results for *Seen*, *Unseen Parameter*, and *Unseen Structure*.

Method	Seen				Unseen Parameter				Unseen Structure			
	STL Sat. $\uparrow$	ρ Mean $\uparrow$	$\rho_{\min}$ $\uparrow$	NL Task $\uparrow$	STL Sat. $\uparrow$	ρ Mean $\uparrow$	$\rho_{\min}$ $\uparrow$	NL Task $\uparrow$	STL Sat. $\uparrow$	ρ Mean $\uparrow$	$\rho_{\min}$ $\uparrow$	NL Task $\uparrow$
Base	41.3	-0.35 ± 1.33	-9.11	90.5	50.0	-0.03 ± 1.26	-5.80	93.9	56.8	0.07 ± 1.51	-6.98	89.3
STL-SFT	61.7	0.49 ± 1.57	-5.09	87.2	59.5	0.34 ± 1.52	-5.05	91.2	64.5	0.71 ± 1.86	-4.64	85.2
Smooth Robust Semantics (1 $\times$)	76.2	1.62 ± 2.92	-5.72	68.5	71.1	1.24 ± 2.51	-4.94	75.9	77.5	2.18 ± 3.26	-4.92	68.6
Smooth Robust Semantics (2 $\times$)	78.8	2.33 ± 3.80	-5.79	45.0	72.1	1.81 ± 3.19	-7.31	47.4	81.8	3.34 ± 4.24	-3.82	42.0
Logic-VLA	82.0	0.81 ± 1.33	-1.62	89.0	74.8	0.89 ± 1.64	-3.79	92.2	82.0	1.24 ± 1.71	-2.46	87.5

adapt the 2B vision–language backbone and 300M action expert using LoRA with ranks 16 and 32 ($\alpha = 16$ and 32), respectively, while fully fine-tuning the vision encoder and action input/output projections throughout the baselines and training procedures. During evaluation, the policy executes 40 actions before re-querying the policy from the latest observation in closed loop at a control frequency of 10 Hz.

Evaluation procedure: Consider an initial state distribution $\mathcal{X}$. A templated STL generation procedure uses a predefined distribution $\mathcal{P}_{\text{STL}}$, from which we sample the formula bank Φ . We instantiate $\mathcal{P}_{\text{STL}}$ from 90 *structural templates* obtained by enumerating temporal compositions $\{F, G, FG, GF, FGF, GFG\}$ and Boolean combinations using $\wedge$ and $\vee$. For each structure, STL formulas are generated by sampling its predicate (within the family of $x, y, z \bowtie c$) and temporal parameters, where $s_t := (x_t, y_t, z_t)$ denotes the drone position. Each temporal interval uses integer endpoints $0 \leq a < b \leq 16$ seconds. Predicates select an axis, a comparison in $\{\leq, \geq\}$, and an integer threshold within the observed coordinate bounds (x : $[-7, 8]$ m; y : $[-10, 16]$ m; z : $[0, 4.6]$ m). We define a *formula-group* as a set of trajectory–formula pairs sharing the same NL task, environment, and STL formula, with trajectory initial states lying in a local neighborhood within the support of $\mathcal{X}$. We use these groups to construct the supervision described in Section IV-A: satisfying trajectories contribute to D^+ , while matched satisfying-violating trajectory pairs contribute to $\mathcal{P}$. This grouping controls task- and environment-induced variation while preserving execution-level diversity.

After filtering and grouping, we obtain 1604 formula-groups generated from 1360 distinct formulas spanning 87 STL structures for training and 3 for unseen structure test. Note that formulas uniformly satisfied or violated are filtered out to remove trivial and infeasible objectives. These groups provide 8886 satisfying trajectory–formula instances, from which we construct the dataset $\mathcal{D}^+$. Among these instances, 6754 admit at least one matched violating counterexample, producing 13494 preference-pair instances in $\mathcal{P}$ for Stage 2. Using datasets $\mathcal{D}^+$ and $\mathcal{P}$, we apply the procedure in Section IV-B and IV-C to obtain policy $\pi_{\hat{\theta}}$. We evaluate $\pi_{\hat{\theta}}$ and baselines under three test settings. Each *evaluation entry* specifies an NL task l , an STL formula ϕ , and a task environment. For each entry, we sample 10 initial states $s_0 \sim \mathcal{X}$ from the corresponding initial-state distribution and execute one closed-loop rollout from each state. *In the first test setting, we evaluate over the STL formulas seen during post-training: Seen* contains 60 evaluation entries sampled from the formula-groups used for post-training. *In the second setting, we test the ability of our policy in generalizing to unseen parameters under STL structures*

used for post-training and STL encoder pre-training: Unseen Parameter contains 99 evaluation entries, covering all 87 STL structures observed during post-training but with newly sampled predicate and temporal parameters held out from training. *In the last setting, we evaluate over STL structures unseen during post-training: Unseen Structure* contains 56 evaluation entries covering 49 STL formulas instantiated from 3 structural templates held out from both post-training and STL encoder pre-training.

Metrics: For each evaluation entry, we execute 10 closed-loop rollouts. Our primary requirement metric is the *STL satisfaction rate*, defined as the fraction of rollouts satisfying the provided specification², i.e., $\rho^\phi(s) \geq 0$. We additionally report the mean STL robust semantics and its standard deviation, as well as the minimum robust semantics $\rho_{\min}$ across all evaluated rollouts, to characterize both average and worst-case requirement satisfaction. To measure preservation of the nominal natural-language task, we report *NL task success*, where a rollout is successful if it reaches all target regions in the prescribed order.

Baselines: All STL-conditioned baselines are initialized from the same STL-blind policy. Their STL graph encoders are initialized from the same pre-trained checkpoint; because the VLM conditioning stream has a different hidden width from encoder pre-training, the token projection is freshly initialized and trained during policy post-training. *a) Base* is the task-adapted STL-blind $\pi_{0.5}$ policy and serves as the initialization for all STL-aware methods. *b) STL-SFT* fine-tunes the base policy on the satisfying demonstrations $\mathcal{D}^+$ using only the standard conditional flow-matching imitation objective described in Sec. IV-C. It tests whether specification-conditioned imitation alone is sufficient to induce requirement-aware behavior. *c) Smooth Robust Semantics* uses the same initialization and $\mathcal{D}^+$ as STL-SFT, but augments the flow-matching objective with direct maximization of differentiable STL robust semantics, where $\mathcal{L}_{\text{smooth}} = \mathcal{L}_{\text{FM}} + \lambda_\rho \mathcal{L}_\rho$ with $\mathcal{L}_\rho = -\mathbb{E}[\tilde{\rho}(\hat{s}, \phi)]$. Here, $\tilde{\rho}$ denotes the differentiable smooth robust semantics [24] evaluated on the reconstructed trajectory $\hat{s}$. We evaluate a nominal robust semantics loss weight λ_ρ (1 $\times$) and a doubled weight (2 $\times$); the two variants are otherwise identical. *d) Logic-VLA* uses STL-SFT as its first stage and subsequently performs trajectory-level preference optimization on the matched satisfying-violating pairs $\mathcal{P}$. Stage 2 optimizes the reference-relative IPO margin together with the preferred-trajectory anchor described in Sec. IV-C.

Requirement Satisfaction and Generalization: The results are shown in Table I, addressing **Q1** and **Q2**. Notice-

²We adopt the convention that zero robust semantics constitutes satisfaction; this differs from the strict positive semantics only when $\rho^\phi(s) = 0$.

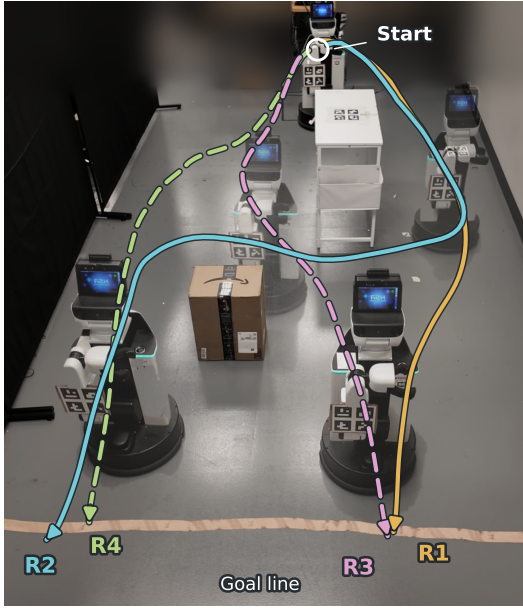


Fig. 3: **Real-robot navigation routes.** Routes R1, R2, R3, and R4 correspond to requirements 1, 2, 3, and 4 in Table IV, respectively. The cabinet is behind the goal line.

TABLE II. **Ablation Results.** a) Encoder pre-training (SFT). STL/Task success (%). b) STL representation (SFT + IPO). STL/Task success (%).

Split	Random	Pre-trained	Text prompt	Encoder
Seen	46.7 / 84.5	61.7 / 87.2	66.5 / 88.5	82.0 / 89.0
Unseen Param.	52.9 / 89.5	59.5 / 91.2	70.1 / 89.7	74.8 / 92.2
Unseen Struct.	56.6 / 83.9	64.5 / 85.2	81.4 / 85.5	82.0 / 87.5

ably, conditioning the policy on STL through SFT already provides a substantial improvement over the STL-blind base policy for all three settings. Direct smooth robust semantics optimization further increases requirement satisfaction, but produces a pronounced trade-off with the nominal NL task. Raising λ_p from $1\times$ to $2\times$ increases seen-specification satisfaction from 76.2% to 78.8%, while task success decreases from 68.5% to 45.0%. The same pattern appears on unseen formulas. This tradeoff is expected: Compared to preference optimization, directly maximizing robust semantics alters the policy behavior beyond what is necessary to satisfy the STL requirements. In all three settings, Logic-VLA achieves the highest observed STL satisfaction while retaining nominal-task success at a level comparable to the base and STL-SFT policies, demonstrating that satisfying-violating sample pairs provide supervision about which executions are favored with information unavailable to positive-only imitation. Across all three settings, Logic-VLA improves satisfaction over the STL-blind base policy by 24.8 to 40.7 percentage points (pp), while the corresponding reduction in task success is at most 1.8 pp. The generalization results show that Logic-VLA responds to the formal structure and parameters of the requirement rather than only memorizing the specifications observed during post-training.

Ablation Studies: To address Q3, we ablate two design choices: semantic initialization of the STL encoder and structured syntax-graph encoding versus direct textual prompting. Table II reports results on *Seen*, *Unseen Parameter*, and

Unseen Structure. a) *Effect of STL encoder pre-training:* Under otherwise matched STL-SFT settings, we compare randomly initialized STL encoders with encoders initialized through robust semantics pre-training (Section IV-B). All encoder parameters remain trainable during policy fine-tuning. Semantic pre-training increases STL satisfaction from 46.7% to 61.7% on seen specifications, from 52.9% to 59.5% on unseen parameters, and from 56.6% to 64.5% on unseen structures. NL task success also improves across all three settings. These results support robust semantics pre-training as a useful initialization for requirement-conditioned policy learning, including generalization to held-out STL structures. b) *Structured STL encoding vs. prompting:* We compare the structured syntax-graph encoder with a baseline that appends a textual rendering of the STL formula to the natural-language instruction. Both methods use the same two-stage post-training procedure. Structured encoding increases STL satisfaction from 66.5% to 82.0% on seen specifications and from 70.1% to 74.8% on unseen parameters. The gain is smaller on unseen structures, where satisfaction increases from 81.4% to 82.0%, while NL task success increases from 85.5% to 87.5%. Overall, structured encoding improves satisfaction and task success across the three settings, with its largest satisfaction advantage on seen specifications.

B. Mobile Robot Navigation

To assess physical deployment, we evaluate Logic-VLA on a Toyota Human Support Robot (HSR) navigating to the cabinets under spatial and temporal requirements. We collect 120 teleoperated demonstrations of the same natural language instruction, “Navigate to the cabinets,” covering all four combinations of passing left or right of a table and a box, with varied travel speeds. The policy is trained on these demonstrations with an 11-formula STL bank and evaluated on the same bank. Figure 3 visualizes route families. Each predicate, such as at table left, holds when the robot’s measured planar position lies within the corresponding bounding box in Table III. The formulas in Table IV impose timed region visits and cabinet-arrival deadlines. All temporal bounds are measured in seconds from episode start. We use s in the temporal bounds to denote seconds. For example, formula 10 requires visiting the table-left region between 3 and 17s and reaching the cabinets within 22s. The policy predicts chunks of 30 waypoints, which a low-level proportional controller tracks at 10 Hz. Inference is asynchronous: the next chunk is requested 0.5s before the current chunk is scheduled to end, allowing inference to overlap with execution. STL satisfaction is evaluated on the measured position trajectory. We evaluate 15 retained trials per formula after excluding collision-triggered emergency-stop trials. Logic-VLA achieves STL satisfaction in 161/165 trials (97.6%). All four violations are temporal and occur in formulas 5, 10, and 11, each of which includes the 22s cabinet-arrival deadline. These three formulas achieve 41/45 satisfaction (91.1%), while the other eight achieve 120/120. End-to-end inference and execution latency may contribute to these timing violations despite asynchronous chunk updates.

TABLE III. Named predicates and their bounding boxes.

Predicate	x interval (m)	y interval (m)
at table left	$[-2.20, -0.92]$	$[0.00, 0.50]$
at table right	$[-0.92, 0.00]$	$[0.00, 0.50]$
at box left	$[-2.60, -1.56]$	$[2.00, 2.50]$
at box right	$[-1.56, -0.10]$	$[2.00, 2.50]$
at cabinets	$[-2.20, -0.20]$	$[3.50, 4.30]$

A predicate is true during training data selection when the robot’s measured position (x_t, y_t) lies inside its listed rectangle. Coordinates are in the world frame, with all boundaries included.

TABLE IV. STL satisfaction on the physical robot.

ID	STL requirement	Satisfied
1	$F_{[3s, 17s]}(\text{at table right}) \wedge F_{[10s, 36s]}(\text{at box right})$	15/15
2	$F_{[3s, 17s]}(\text{at table right}) \wedge F_{[10s, 36s]}(\text{at box left})$	15/15
3	$F_{[3s, 17s]}(\text{at table left}) \wedge F_{[10s, 36s]}(\text{at box right})$	15/15
4	$F_{[3s, 17s]}(\text{at table left}) \wedge F_{[10s, 36s]}(\text{at box left})$	15/15
5	$F_{[0s, 22s]}(\text{at cabinets})$	14/15
6	$F_{[3s, 17s]}(\text{at table left})$	15/15
7	$F_{[3s, 17s]}(\text{at table right})$	15/15
8	$F_{[10s, 36s]}(\text{at box left})$	15/15
9	$F_{[10s, 36s]}(\text{at box right})$	15/15
10	$F_{[3s, 17s]}(\text{at table left}) \wedge F_{[0s, 22s]}(\text{at cabinets})$	13/15
11	$F_{[10s, 22s]}(\text{at box right}) \wedge F_{[0s, 22s]}(\text{at cabinets})$	14/15
Overall		161/165

Predicate bounds are given in Table III. The table and box are randomly placed in distribution during evaluation. Positional satisfactions are measured from observing left vs. right of each asset. Overall satisfaction is 97.6%.

Among retained trials, Logic-VLA achieves high satisfaction across the tested STL requirements, demonstrating the feasibility of STL-conditioned navigation on a physical robot.

VI. CONCLUSION

We propose Logic-VLA, a temporal logic conditioned VLA. Logic-VLA combines a semantically pre-trained syntax-graph encoder with two-stage post-training using satisfying demonstrations and matched satisfying-violating trajectory pairs. In closed-loop quadcopter navigation, it substantially improves STL satisfaction while minimally reducing NL task success compared to an STL-blind base policy. The results show that formal logic can serve as effective conditioning signals for adapting a single VLA to varying requirements, including safety-critical and spatiotemporal specifications.

REFERENCES

- [1] P. Intelligence, K. Black, N. Brown *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [2] B. Zitkovich, T. Yu, S. Xu *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*. PMLR, 2023.
- [3] O. Maler and D. Nickovic, “Monitoring temporal properties of continuous signals,” in *FTRFT*. Springer, 2004, pp. 152–166.
- [4] G. E. Fainekos and G. J. Pappas, “Robustness of temporal logic specifications for continuous-time signals,” *Theoretical Computer Science*, 2009.
- [5] A. Puranic, J. Deshmukh, and S. Nikolaidis, “Learning from demonstrations using signal temporal logic,” in *CoRL*. PMLR, 2021.
- [6] Y. Meng and C. Fan, “Telograf: Temporal logic planning via graph-encoded flow matching,” *arXiv preprint arXiv:2505.00562*, 2025.

- [7] M. G. Azar, Z. D. Guo, B. Piot *et al.*, “A general theoretical paradigm to understand learning from human preferences,” in *AISTATS*. PMLR, 2024.
- [8] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [9] Q. Li, B. Yin, W. Huang *et al.*, “Vision-language-action safety: Threats, challenges, evaluations, and mechanisms,” *arXiv preprint arXiv:2604.23775*, 2026.
- [10] B. Zhang, Y. Zhang, J. Ji *et al.*, “Safevla: Towards safety alignment of vision-language-action model via constrained learning,” *NeurIPS*, vol. 38, 2026.
- [11] J. Fan, W. Xu, O. Sokolsky *et al.*, “Safevla-bench: A benchmark for the success-safety gap in vision-language-action models,” *arXiv preprint arXiv:2606.00773*, 2026.
- [12] S. Hu, Z. Liu, S. Liu *et al.*, “Vlsa: Vision-language-action models with plug-and-play safety constraint layer,” *arXiv preprint arXiv:2512.11891*, 2025.
- [13] W. English, H. Zheng, and R. Ewetz, “Neuro-symbolic safety guidance for vision-language-action models via constrained flow matching,” *arXiv preprint arXiv:2607.01378*, 2026.
- [14] Y. Meng, M. Xia, and D. Chen, “Simplo: Simple preference optimization with a reference-free reward,” *NeurIPS*, 2024.
- [15] S. Liu, W. Fang, Z. Hu *et al.*, “A survey of direct preference optimization,” *arXiv preprint arXiv:2503.11701*, 2025.
- [16] R. Rafailov, A. Sharma, E. Mitchell *et al.*, “Direct preference optimization: Your language model is secretly a reward model,” *NeurIPS*, vol. 36, pp. 53 728–53 741, 2023.
- [17] R. Tian, Y. Wu, C. Xu *et al.*, “Maximizing alignment with minimal feedback: Efficiently learning rewards for visuomotor robot policy alignment,” *arXiv preprint arXiv:2412.04835*, 2024.
- [18] Y. Chen, D. K. Jha, M. Tomizuka, and D. Romeres, “Fdpp: Fine-tune diffusion policy with human preference,” in *ICRA*, 2025.
- [19] Z. Zhang, K. Zheng, Z. Chen *et al.*, “Grape: Generalizing robot policy via preference alignment,” *arXiv preprint arXiv:2411.19309*, 2024.
- [20] Y. Wu, H. Zhang, J. Tan *et al.*, “Flowpro: Reward-free reinforced fine-tuning of flow-matching vls via proximalized preference optimization,” *arXiv preprint arXiv:2606.05468*, 2026.
- [21] M. Torne, A. Mahajan, A. Bhat, and C. Finn, “Freeform preference learning for robotic manipulation,” *arXiv preprint arXiv:2606.32027*, 2026.
- [22] A. Donzé and O. Maler, “Robust satisfaction of temporal logic over real-valued signals,” in *FORMATS*. Springer, 2010, pp. 92–106.
- [23] V. Raman, A. Donzé, M. Maasoumy *et al.*, “Model predictive control with signal temporal logic specifications,” in *CDC*. IEEE, 2014.
- [24] Y. V. Pant, H. Abbas, and R. Mangharam, “Smooth operator: Control using the smooth robustness of temporal logic,” in *CCTA*. IEEE, 2017.
- [25] Z. Guo, W. Zhou, and W. Li, “Temporal logic specification-conditioned decision transformer for offline safe reinforcement learning,” *arXiv preprint arXiv:2402.17217*, 2024.
- [26] B. Ye, J. Huang, Y. Liu *et al.*, “Bridging perception and planning: Towards end-to-end planning for signal temporal logic tasks,” *arXiv preprint arXiv:2509.12813*, 2025.
- [27] Z. Liu, Z. Zheng, H. Luo *et al.*, “Vision-tl-action: Neuro-symbolic trajectory generation from visual observations and temporal logic,” *arXiv preprint arXiv:2607.26770*, 2026.
- [28] M. Zoellner, A. Manganaris, and R. Paleja, “Temporal logic guidance for action-only diffusion policies with world models,” *arXiv preprint arXiv:2606.22729*, 2026.
- [29] K. Torshizi, A. Singh, S. Mathur *et al.*, “Step: Signal temporal logic for precise specifications for action generation with vision language models,” *arXiv preprint arXiv:2607.18580*, 2026.
- [30] L. Lindemann and D. V. Dimarogonas, “Control barrier functions for signal temporal logic tasks,” *L-CSS*, 2018.
- [31] Y. Zhao, T. Kim, H. Okamoto *et al.*, “Safety-aware model predictive path integral control with signal temporal logic,” *arXiv preprint arXiv:2608.23972*, 2026.
- [32] S. Sadraddini and C. Belta, “Robust temporal logic model predictive control,” in *Allerton*. IEEE, 2015, pp. 772–779.
- [33] D. Ničković and T. Yamaguchi, “Rtam: Online robustness monitors from stl,” in *ATVA*. Springer, 2020, pp. 564–571.
- [34] D. McAllister, S. Ge, B. Yi *et al.*, “Flow matching policy gradients,” in *ICLR*, 2026.